



MINI-PROJECT REPORT

Project Report: Traffic Controller

Student Name: Jatin Tasoria

UID: 25MCC20054

Branch: MCA-CCD

Section/Group:102/A

Semester: Ist

Date: 30/10/2025

Subject Name: Python Programming

Subject Code: 25CAH-606

Submitted To:

Harmanjot Kaur_E19510

BONAFIDE CERTIFICATE

This is to certify that the mini project report entitled “Traffic Controller” submitted by Mr./Ms. Jatin Tasoria to the Department of Computer Science and Engineering in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA) is a record of the original work carried out by him/her under my supervision and guidance. The project work embodied in this report has not been submitted previously, in part or full, to any other University or Institution for the award of any degree or diploma.

Project Guide (Signature)

Name: Ms Harmanjot Kaur_E19510

Department of Computer Science & Engineering

Place: _____

Date: _____

Acknowledgement

I would like to express my deepest gratitude to all those who have supported me throughout the completion of my mini project titled “Traffic Controller”. This project has been a wonderful learning experience that has enhanced my understanding of Python programming, database connectivity, and algorithm implementation.

I wish to express my heartfelt thanks to my project guide, Ms. Harmanjot Kaur, and the faculty members for their invaluable guidance, continuous encouragement, and constructive feedback during every stage of this project. Their insights have been instrumental in shaping the final outcome.

I am sincerely thankful to my institution for providing facilities and a learning environment that made this project possible. A special thanks to my friends, classmates, and family for their continuous support, motivation, and encouragement throughout this journey.

This project marks not only the successful completion of an academic requirement but also a significant milestone in my personal and professional growth. Through this work, I have gained practical knowledge and confidence in integrating algorithmic logic with database systems and visualization tools, which will be immensely useful in my future career endeavors.

Table of Contents

1. Acknowledgement
2. Abstract
3. Introduction
4. Objectives
5. System Requirements
6. Literature Review
7. System Design
8. Algorithm Used (Dijkstra's Algorithm)
9. Database Design
10. Project Code
11. Output Screens (Bar Chart & Node Graph)
12. Results and Discussion
13. Learning Outcomes

Abstract

The Traffic Controller is a Python and PostgreSQL-based mini-project designed to simulate a traffic management system capable of identifying the fastest route between intersections. Using Dijkstra's Algorithm, the project calculates the shortest travel time considering factors such as distance, speed limit, and traffic density. The visualization is implemented using Matplotlib to display both a bar chart of segment-wise travel time and a complete node graph of the traffic network.

The system reads intersection and road data stored in a PostgreSQL database, constructs a weighted graph, and then applies Dijkstra's algorithm to find the optimal path. The results are displayed both textually and graphically, helping understand how routing algorithms and graph theory can be applied in real-world traffic systems.

This project successfully demonstrates how algorithmic concepts can be integrated with database applications to create meaningful, real-world solutions that assist users in achieving better financial awareness and control.

3. Introduction

The Traffic Controller project is a demonstration of how shortest path algorithms like Dijkstra's can be used in real-world transportation and navigation systems. By integrating Python, PostgreSQL, and Matplotlib, the system efficiently computes and visualizes the best route between two intersections while also providing analytical insights.

4. Objectives

- To develop a Python-based system for route optimization using Dijkstra's Algorithm.
- To store and retrieve intersection and road data from a PostgreSQL database.
- To visualize travel times using bar and network graphs.
- To analyze and highlight the importance of algorithmic efficiency in traffic control systems.

5. System Requirements

Hardware Requirements:

- Processor: Intel Core i3 or higher
- RAM: Minimum 4 GB
- Storage: Minimum 500 MB free space
- OS: Windows 10 or Linux Mint

Software Requirements:

- Programming Language: Python 3.8+
- Database: PostgreSQL
- Libraries: psycpg2, matplotlib
- IDE: VS Code / PyCharm

6. Literature Review

Several routing and navigation systems exist today, such as Google Maps and Waze, that use complex shortest path algorithms. This project focuses on implementing a simplified academic

version using Dijkstra's algorithm to demonstrate the relationship between graph theory and real-time route computation. The project also integrates data visualization using Matplotlib, allowing users to visually understand how routes and traffic density affect travel time.

7. System Design

Architecture: The system follows a three-tier architecture:

1. Presentation Layer – Python console interface and Matplotlib visualizations.
2. Logic Layer – Dijkstra's Algorithm for route optimization.
3. Data Layer – PostgreSQL database for storing intersections and roads.

Data Flow:

User selects start and destination → System fetches road data → Dijkstra's algorithm computes fastest path → Results visualized.

8. Algorithm Used (Dijkstra's Algorithm)

Dijkstra's algorithm is used to find the shortest path between nodes in a weighted graph. In this project, weights represent travel time calculated as $(\text{distance} / \text{speed}) \times \text{traffic_factor}$.

1. Initialize all node distances as infinity except the start node (set to 0).
2. Visit the unvisited node with the smallest distance value.
3. Update the distances of its neighboring nodes.
4. Mark the current node as visited.
5. Repeat until the destination node is reached.

9. Database Design

```
CREATE TABLE intersections (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE roads (  
    id SERIAL PRIMARY KEY,  
    source_id INT REFERENCES intersections(id) ON DELETE CASCADE,  
    destination_id INT REFERENCES intersections(id)  
ON DELETE CASCADE,  
    distance FLOAT NOT NULL,  
    traffic_factor FLOAT DEFAULT 1.0,  
    speed_limit INT DEFAULT 60  
);  
  
-- Sample Data  
INSERT INTO intersections (name) VALUES  
('A'), ('B'), ('C'), ('D'), ('E');  
  
INSERT INTO  
    roads (source_id, destination_id, distance, traffic_factor, speed_  
limit)  
VALUES  
(1, 2, 5.0, 1.2, 60),  
(2, 3, 3.0, 1.5, 50),  
(1, 3, 10.0, 1.0, 80),  
(3, 4, 2.0, 1.1, 40),  
(4, 5, 4.0, 1.0, 70),  
(2, 5, 12.0, 1.3, 60);
```

10. Project Code

The project includes the following files:

- config.py – PostgreSQL connection setup.
- dijkstra.py – Implementation of Dijkstra's algorithm.
- chart.py – Visualization using Matplotlib.
- main.py – Main execution logic combining all modules.

config.py

```
import psycopg2

def get_connection():
    return psycopg2.connect(
        dbname="traffic_db",
        user="postgres",
        password="Jatin",
        host="localhost",
        port="5432"
    )
```

Dijkstra.py

```
import heapq

def dijkstra(graph, start, end):
    queue = [(0, start, [])]
    visited = set()

    while queue:
        (cost, node, path) = heapq.heappop(queue)
        if node in visited:
            continue

        path = path + [node]
        visited.add(node)

        if node == end:
            return cost, path

        for (adj, weight) in graph.get(node, []):
            if adj not in visited:
                heapq.heappush(queue, (cost +
weight, adj, path))
    return float("inf"), []
```

chart.py

```
● ● ●

# chart.py
import matplotlib.pyplot as plt
import math
from config import get_connection

def show_route_visuals(path):
    conn = get_connection()
    cur = conn.cursor()
    # --- Fetch intersections and roads from DB ---
    cur.execute("SELECT id, name FROM intersections;")
    intersections = cur.fetchall()
    cur.execute("""
        SELECT
            s.id AS source_id,
            d.id AS dest_id,
            s.name || ' - ' || d.name AS road,
            r.distance,
            r.speed_limit,
            r.traffic_factor
        FROM roads r
        JOIN intersections s ON r.source_id = s.id
        JOIN intersections d ON r.destination_id = d.id;
    """)
    all_roads = cur.fetchall()
    conn.close()
    if not all_roads or not intersections:
        print("No data available to visualize.")
        return
    # --- Selected route edges ---
    selected_roads = []
    if path and len(path) >= 2:
        for i in range(len(path) - 1):
            a, b = path[i], path[i + 1]
            for road in all_roads:
                if (road[0] == a and road[1] == b) or (road[0] == b and road[1] == a):
                    selected_roads.append(road)
                    break
    # --- Bar chart data for selected route ---
    road_names = [r[2] for r in selected_roads]
    travel_times = [(r[3] / r[4]) * r[5] for r in selected_roads]
    if selected_roads else []
    # --- Figure setup (2 charts side by side) ---
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 7))
    fig.suptitle("Traffic Control Visualization", fontsize=15, fontweight="bold")
    # ----- BAR CHART -----
    if selected_roads:
        bars = ax1.bar(road_names, travel_times, color="skyblue", width=0.5)
        for bar, time in zip(bars, travel_times):
            ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height()/2, f"{time:.2f} h", ha="center", va="center", fontsize=10)
        ax1.set_xlabel("Road Segments")
```

```

ax1.set_ylabel("Travel Time (hours)")
ax1.set_title("Selected Route Travel Times")
else:
    ax1.text(0.5, 0.5, "No Route Selected", ha="center", va=
"center",
            fontsize=12, color="gray")
    ax1.axis("off")
ax1.grid(axis="y", linestyle="--", alpha=0.7)

# ----- NODE GRAPH -----
n = len(intersections)
positions = {}
for i, (id, name) in enumerate(intersections):
    # Simple circular layout for visualization
    angle = 2 * math.pi * i / n
    x = 10 * math.cos(angle)
    y = 10 * math.sin(angle)
    positions[id] = (x, y)
ax2.set_title("All Intersections and Roads (with Travel Times)")
ax2.axis("off")
# --- Draw all roads (gray lines + time labels) ---
for road in all_roads:
    s, d, _, dist, speed, traffic = road
    x1, y1 = positions[s]
    x2, y2 = positions[d]
    time = (dist / speed) * traffic
    ax2.plot([x1, x2], [y1, y2], color="lightgray", linestyle="--",
, linewidth=1)
    mid_x, mid_y = (x1 + x2)/2, (y1 + y2)/2
    ax2.text(mid_x, mid_y, f"<time:.2f>h", color="gray", fontsize=8
, ha="center", va="center")
# --- Highlight selected route (red edges + red labels) ---
if selected_roads:
    for road in selected_roads:
        s, d = road[0], road[1]
        x1, y1 = positions[s]
        x2, y2 = positions[d]
        time = (road[3] / road[4]) * road[5]
        ax2.plot([x1, x2], [y1, y2], color="red", linewidth=3,
zorder=5)
        mid_x, mid_y = (x1 + x2)/2, (y1 + y2)/2
        ax2.text(mid_x, mid_y, f"<time:.2f>h", color="red",
fontsize=9,
                fontweight="bold", ha="center", va="center")

# --- Draw all intersections (nodes) ---
for id, name in intersections:
    x, y = positions[id]
    ax2.scatter(x, y, color="skyblue", s=300, edgecolors="black",
zorder=10)
    ax2.text(x, y, name, ha="center", va="center", fontsize=10,
fontweight="bold")

plt.tight_layout()
plt.show()

```

main.py

```
from config import get_connection
from dijkstra import dijkstra
from chart import show_route_visuals

def load_graph():
    conn = get_connection()
    cur = conn.cursor()

    cur.execute("SELECT id, name FROM intersections;")
    intersections = {i[0]: i[1] for i in cur.fetchall()}

    cur.execute(
        "SELECT source_id, destination_id, distance, traffic_factor, speed_
        limit FROM roads;"
    )
    roads = cur.fetchall()

    graph = {}
    for s, d, dist, traffic, speed in roads:
        travel_time = (dist / speed) * traffic
        graph.setdefault(s, []).append((d, travel_time))
        graph.setdefault(d, []).append((s, travel_time))

    cur.close()
    conn.close()
    return graph, intersections

def main():
    graph, intersections = load_graph()

    print("\nAvailable Intersections:")
    for k, v in intersections.items():
        print(f"{k}: {v}")

    start = int(input("\nEnter Start Intersection ID: "))
    end = int(input("Enter Destination Intersection ID: "))

    total_time, path = dijkstra(graph, start, end)

    if path:
        names = " → ".join(intersections[i] for i in path)
        print(f"\nFastest Path: {names}")
        print(f"Estimated Travel Time: {total_time:.2f} hours")

        print("\nGenerating combined visualization...")
        show_route_visuals(path)
    else:
        print("No available path found.")

if __name__ == "__main__":
    main()
```

13. Results and Discussion

The system successfully computes and visualizes the fastest route using Dijkstra's algorithm. The bar chart provides a clear comparison of travel times across road segments, while the network graph gives a comprehensive overview of the entire road system, highlighting the optimal route.

14. Learning Outcomes

From this project, I have learnt:

- Implementation of Dijkstra's algorithm for real-world traffic systems.
- Database integration with PostgreSQL.
- Visualization of data using Matplotlib.
- Modular Python project structuring.
- Practical application of graph theory concepts in traffic management.